N051 한 번 만든 이미지로 배포하기_해설서

Chapter01

/강의정리 챕터 01. 한 번 만든 이미지로 배포하기

1. 강의 목적

이번 강의의 핵심은 한 번 만든 Docker 이미지를 staging에서 검증하고, 검증된 바로 그 이미지를 production에 배포하는 것입니다.

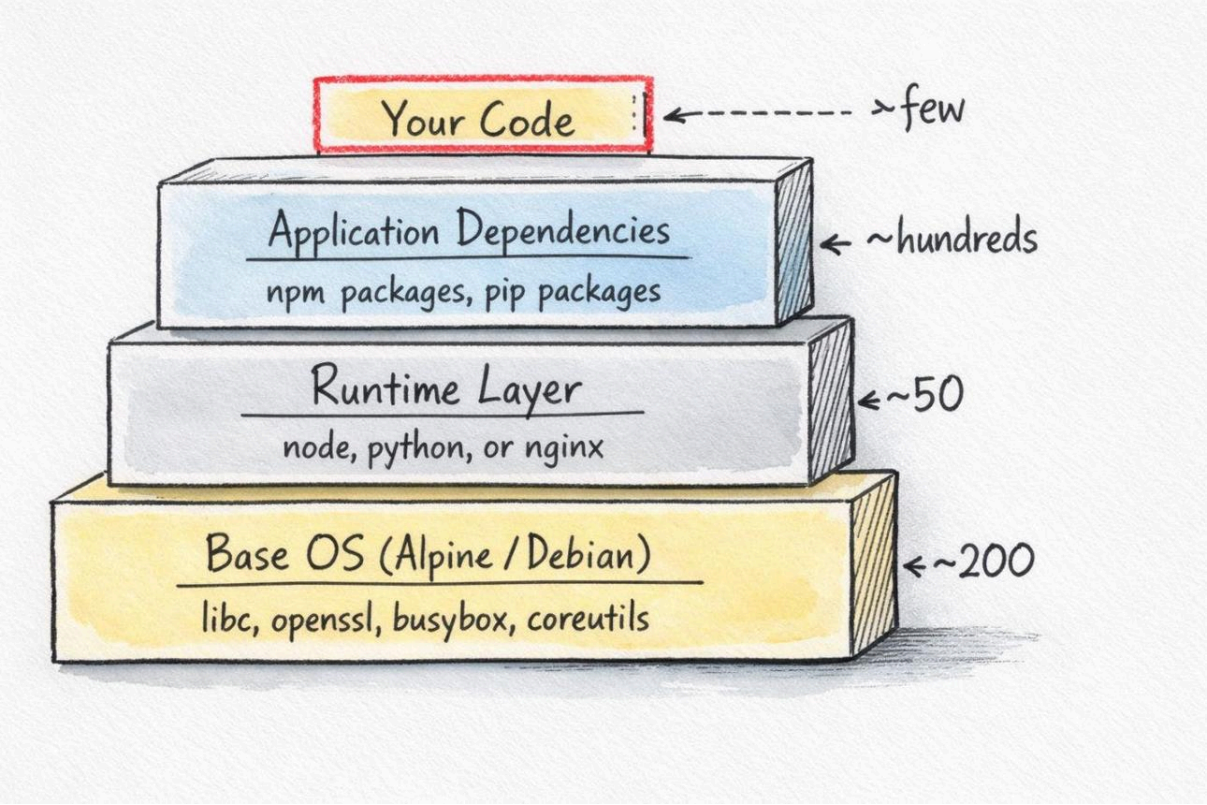
AI 에이전트 서비스를 자동으로 배포하는 것만으로는 충분하지 않습니다. 실제 운영 중 문제가 생겼을 때 다음 세 가지 질문에 답할 수 있어야 합니다.

- 추적성: 지금 운영 중인 코드는 어느 커밋에서 만들어졌습니까?
- 복구성: 문제가 생기면 정상적으로 동작하던 이전 버전으로 되돌릴 수 있습니까?
- 동일성: staging에서 검증한 이미지와 production에서 실행 중인 이미지가 정말 같습니까?

이를 위해 GitHub Actions, Docker, Artifact Registry, Cloud Run을 연결하고 커밋부터 실제 실행 버전까지 추적하는 구조를 학습합니다.

2. 중학생도 이해할 수 있는 설명

2-1. Docker 이미지는 제품의 완성된 포장 상자입니다



hackernoon.com

AI 에이전트 서비스를 음식점이라고 생각해 보겠습니다.

- 코드: 요리 레시피
- Dockerfile: 재료를 준비하고 요리하는 방법을 적은 조리법
- Docker 이미지: 레시피와 재료를 포함해 실행할 수 있도록 완성한 제품 상자
- 컨테이너: 이미지로 실제 실행한 프로그램
- Cloud Run: 완성된 제품을 실행해 사용자에게 제공하는 장소

staging은 제품을 판매하기 전에 품질을 검사하는 시험 매장이고, production은 실제 고객이 이용하는 정식 매장입니다.

시험 매장에서 확인한 제품을 다시 만들면 재료나 조리 과정이 달라질 가능성이 있습니다. 따라서 검증을 마친 완성품 자체를 정식 매장으로 옮기는 것이 이번 강의의 핵심입니다.

2-2. 전체 배포 흐름

① GitHub 커밋 / PR 머지

변경된 코드와 커밋 SHA 확정

② GitHub Actions CI

자동 테스트로 코드 검증

③ Docker 이미지 빌드

커밋 SHA를 태그로 지정

④ Artifact Registry에 저장

이미지와 다이제스트 관리

⑤ Staging 배포 및 검증

실제 이미지가 정상적으로 작동하는지 확인

⑥ 동일한 이미지로 Production 배포

검증된 다이제스트를 그대로 사용

여기서 Artifact Registry는 완성된 이미지들을 보관하는 창고입니다. Cloud Run은 그 창고에서 이미지를 가져와 서비스를 실행합니다.

주의할 점은 PR 머지, 이미지 빌드, staging 배포, production 배포가 반드시 하나의 워크플로에서 모두 자동으로 실행된다는 뜻은 아니라는 것입니다. 실제 자동화 방식은 GitHub Actions의 워크플로 설정에 따라 달라집니다.

3. 꼭 이해해야 할 핵심 개념

3-1. 이미지 이름, 태그, 다이제스트

예를 들어 다음과 같은 이미지가 있다고 하겠습니다.

```
asia-northeast3-docker.pkg.dev/my-project/agents/ai-friend:a1b2c3d
```

구성 요소	의미
`asia-northeast3-docker.pkg.dev`	이미지 저장소 주소
`my-project`	Google Cloud 프로젝트
`agents`	저장소 이름
`ai-friend`	이미지 이름
`a1b2c3d`	커밋 SHA를 이용한 태그

태그는 이미지에 붙이는 이름표입니다. 다이제스트는 이미지의 실제 내용에 대응하는 고유한 식별값입니다.

예를 들어 택배 상자를 생각하면 이해하기 쉽습니다.

- 태그: `정상제품`, `10월 27일 제품` 같은 이름표
- 다이제스트: 상자의 내용물을 식별하는 고유한 지문

태그는 다른 이미지에 다시 연결될 수 있지만, 다이제스트는 해당 이미지 내용을 식별하는 데 사용됩니다. 따라서 배포할 이미지를 정확히 지정하려면 다이제스트를 사용하는 편이 안전합니다.

3-2. 왜 `latest`만 사용하면 안 될까요?

`latest`는 흔히 최신 버전이라는 의미로 쓰이는 태그입니다. 하지만 이미지가 실제로 언제 만들어졌는지, 어느 커밋에서 만들어졌는지를 이름만으로 확실하게 알 수는 없습니다.

예를 들어 다음과 같습니다.

- 월요일 이미지: `ai-friend:latest`
- 화요일 이미지: `ai-friend:latest`

화요일에 새 이미지가 같은 태그를 사용하면 태그가 가리키는 대상이 바뀔 수 있습니다.

반면 커밋 SHA를 사용하면 다음과 같이 구분할 수 있습니다.

- `ai-friend:a1b2c3d`
- `ai-friend:e4f5g6h`

각 태그를 통해 어떤 코드 버전에서 이미지를 만들었는지 추적하기가 쉬워집니다.

기억할 점: 커밋 SHA 태그는 추적성을 높이고, 다이제스트는 실제 이미지의 동일성을 확인하는 데 도움을 줍니다.

단, 태그가 있다고 해서 그 태그가 가리키는 이미지가 절대로 바뀌지 않는 것은 아닙니다. 태그 덮어쓰기 방지 정책도 중요합니다.

3-3. 커밋에서 실행 중인 리비전까지 연결하기

이번 강의에서 확인하려는 정보는 다음과 같습니다.

Git 커밋 SHA

어떤 코드 변경 사항인가?

Docker 이미지 태그 / 다이제스트

어떤 이미지로 만들었는가?

Cloud Run 리비전

어떤 이미지가 배포되었는가?

`/version` 응답

실행 중인 서비스가 어떤 버전이라고 보고하는가?

이 정보들을 연결하면 장애가 발생했을 때 어느 코드가 문제인지 조사하기 쉬워집니다.

여기서 `/version`은 서비스가 버전 정보를 반환하도록 직접 구현해야 하는 엔드포인트입니다. 예를 들어 커밋 SHA를 반환하도록 만들 수 있습니다. 다만 `/version`이 반환하는 정보만으로 실제 실행 이미지의 동일성이 완전히 증명되는 것은 아니므로, Cloud Run 리비전의 이미지 정보도 함께 확인해야 합니다.

4. 실제 업무에 적용하는 예시

가령 AI 친구 에이전트가 약속 알림을 잘못 보내는 문제가 발생했다고 가정하겠습니다.

1. Cloud Run 콘솔에서 production의 현재 리비전을 확인합니다.
2. 해당 리비전이 사용하는 이미지와 다이제스트를 확인합니다.
3. 이미지 태그 또는 빌드 기록을 통해 커밋 SHA를 추적합니다.
4. `/version` 응답과 대조해 서비스가 보고하는 버전을 확인합니다.
5. 직전 정상 버전의 이미지가 남아 있다면 해당 이미지로 되돌리는 방법을 검토합니다.
6. staging과 production의 다이제스트를 비교해 동일한 이미지인지 확인합니다.

이 과정을 거치면 단순히 "최근에 코드를 수정했으니 그게 원인일 것"이라고 추측하는 대신, 실제 운영 환경의 증거를 기반으로 조사할 수 있습니다.

롤백을 위해 기억할 점

이전 리비전이 남아 있더라도 그 리비전이 사용하는 이미지가 삭제되었다면 재배포에 문제가 생길 수 있습니다. 따라서 이미지 정리 정책을 만들 때는 최신 이미지뿐 아니라 실제로 롤백에 필요한 이전 이미지도 보존해야 합니다.

5. 로컬에서 만든 이미지가 Cloud Run에서 실행되지 않는 이유

내 컴퓨터에서 정상적으로 실행되는 이미지라고 해서 Cloud Run에서도 반드시 실행되는 것은 아닙니다.

대표적인 원인은 다음과 같습니다.

- CPU 아키텍처가 다릅니다. 예를 들어 로컬은 ARM, 배포 환경은 AMD64일 수 있습니다.
- 환경 변수나 API 키 등 실행에 필요한 설정이 다릅니다.
- 운영 환경에 필요한 파일이나 라이브러리가 이미지에 포함되지 않았습니다.
- 컨테이너가 Cloud Run에서 요구하는 방식으로 포트를 열거나 서비스를 시작하지 않습니다.

따라서 운영 배포용 이미지는 정해진 빌드 환경과 절차를 통해 만들고, 동일한 이미지를 staging과 production에서 사용하는 것이 좋습니다.

6. AI 개발자와 1인 AI 사업가가 알아야 하는 이유

사용자께서 준비 중인 AI 컨설팅 사업이나 My-Story-Maker 같은 AI 서비스를 운영한다고 생각해 보겠습니다.

초기에는 혼자 개발하고 직접 테스트하더라도, 서비스가 고객에게 제공되기 시작하면 장애 대응과 변경 이력 관리가 중요해집니다.

운영 상황	이번 강의에서 얻는 실무적 이점
고객이 오류를 신고함	실제 운영 버전과 변경 이력을 빠르게 추적
새 기능 배포 후 오류 발생	이전 이미지로 복구할 수 있도록 준비
staging 테스트는 성공했지만 운영에서 오류 발생	두 환경이 같은 이미지를 사용했는지 확인
이미지가 계속 쌓임	보존 기준과 정리 정책으로 저장 공간 관리
외부 개발자와 협업	커밋, 이미지, 리비전을 기준으로 명확하게 소통

1인 사업에서는 개발 인력이 적기 때문에 문제가 생겼을 때 원인을 찾고 복구하는 데 드는 시간이 사업 운영에 직접적인 영향을 줍니다. 배포 이력을 체계적으로 관리하면 이러한 부담을 줄일 수 있습니다.

7. 오늘의 학습 목표 체크리스트

학습 점검

0/8 완료

- [ ] 이미지 이름의 구성 요소를 설명할 수 있다.
- [ ] 태그와 다이제스트의 차이를 설명할 수 있다.
- [ ] 커밋 SHA 태그가 필요한 이유를 설명할 수 있다.
- [ ] GitHub Actions에서 이미지 빌드와 업로드 과정을 이해한다.
- [ ] 커밋부터 Cloud Run 리비전까지 추적할 수 있다.
- [ ] staging과 production의 다이제스트를 비교할 수 있다.
- [ ] 이전 버전의 이미지 보존과 롤백 정책을 이해한다.
- [ ] Dockerfile 오류의 원인을 로그와 증상으로 추적할 수 있다.

8. 반드시 기억할 핵심 정리

- Dockerfile은 이미지를 만드는 방법을 정의합니다.
- Docker 이미지는 실행에 필요한 내용을 담은 패키지입니다.
- 태그는 이름표이고, 다이제스트는 이미지 내용을 식별하는 고유한 값입니다.
- 커밋 SHA 태그를 사용하면 코드 변경과 이미지의 연결을 추적하기 쉽습니다.

- staging에서 검증한 이미지를 다시 빌드하지 않고 production에 배포하면 두 환경의 이미지 동일성을 확보하기 쉽습니다.
- Cloud Run 리비전과 `/version`을 확인하고 이미지 다이제스트까지 대조해야 실제 운영 버전을 제대로 추적할 수 있습니다.
- 롤백에 필요한 이미지는 정리 정책에서 보호해야 합니다.

한 문장 요약

이번 강의는 코드를 배포하는 것에서 한 걸음 더 나아가, 어떤 코드로 만든 이미지가 실제 운영 중인지 추적하고 검증된 동일한 이미지로 안전하게 배포·복구하는 방법을 배우는 과정입니다.

Chapter02

/강의정리 챕터 02. 시작 상태 맞추기

1. 강의 목적

이번 강의의 핵심은 이미지 배포 실습을 시작하기 전에 개발 환경을 점검하고, Docker 이미지의 다이제스트를 이용해 실제 운영 버전을 정확히 식별하는 것입니다.

챕터 01에서 배운 것이 이미지를 한 번 만들어 여러 환경에 배포하는 원칙이었다면, 이번 챕터에서는 그 원칙이 왜 필요한지 실제 운영 상황을 통해 확인합니다.

이번 강의에서 해결하려는 문제는 세 가지입니다.

1. 지금 production에서 실행 중인 코드는 어느 커밋인가?
2. 문제가 발생하면 정상적으로 동작하던 이전 버전으로 되돌릴 수 있는가?
3. staging에서 검증한 이미지와 production의 이미지가 정말 같은가?

2. 중학생도 이해할 수 있는 설명

2-1. 실습 시작 상태는 공사를 시작하기 전의 현장 점검입니다

집을 짓기 전에 전기와 수도가 연결되어 있는지, 자재가 준비되어 있는지 확인하는 것과 같습니다.

AI 에이전트 서비스를 배포하려면 다음 준비물이 필요합니다.

준비 항목	쉬운 설명
Google Cloud 프로젝트	작업할 클라우드 공간
`gcloud` 로그인과 API	클라우드에 접속하고 서비스를 이용할 수 있는 상태
`friend-agent` 서비스	기존 운영 서비스
GitHub Actions 배포 권한(WIF)	GitHub가 Google Cloud에 안전하게 접근하는 통로
CI 및 배포 워크플로	자동 검사와 배포를 수행하는 작업 지침
`friend-agent-staging`	운영 전 테스트 공간
`Dockerfile`과 `.dockerignore`	이미지 제작 방법과 제작에서 제외할 파일 목록

중요: 준비 상태를 확인하는 것과 새로 만들거나 변경하는 것은 별개입니다. 강의에서 안내하는 시작 상태 확인 명령은 조회만 수행하며, 빠진 항목은 문서의 A~I 단계에 따라 필요한 것부터 준비합니다.

2-2. 다이제스트는 이미지의 지문입니다



태그(Tag) = 상자에 붙인 이름표

`latest`, `a1b2c3d`처럼 사람이 알아보기 쉽게 붙인 이름입니다. 이름표는 다른 이미지로 옮겨 붙일 수 있습니다.



다이제스트(Digest) = 이미지의 지문

`sha256:0d01a69...`처럼 이미지 내용을 식별하는 값입니다. 정확히 어떤 이미지인지 확인할 때 사용합니다.

비유로 정리하면 다음과 같습니다.

- 태그가 같다고 해서 이미지가 같은 것은 아닙니다.
- 태그가 바뀌어도 기존에 배포한 Cloud Run 리비전이 자동으로 새 이미지로 바뀌지는 않습니다.
- 다이제스트가 같으면 해당 이미지의 내용이 같다는 것을 확인할 수 있습니다.

2-3. 이미지 이름 읽는 법

예를 들어 다음 이미지 주소가 있다고 하겠습니다.

`us-central1-docker.pkg.dev/my-project/friend-agent/friend-agent:a1b2c3d`

부분	의미
`us-central1-docker.pkg.dev`	Artifact Registry 주소와 리전

부분	의미
`my-project`	Google Cloud 프로젝트 ID
첫 번째 `friend-agent`	이미지 저장소 이름
두 번째 `friend-agent`	이미지 이름
`a1b2c3d`	커밋 SHA를 이용한 태그

이미지의 이름표에 Git 커밋 정보가 들어가면 어떤 코드로 이미지를 만들었는지 찾기가 쉬워집니다.

정확한 이미지 하나를 지정하려면 태그 대신 다음처럼 다이제스트를 사용할 수 있습니다.

`us-central1-docker.pkg.dev/my-project/friend-agent/friend-agent@sha256:...`

3. 현재 배포 방식의 문제점

3-1. 운영 버전을 찾는 데 시간이 걸립니다

기존 구조에서는 Cloud Build가 이미지를 만들고 Artifact Registry에 저장합니다. 이미지에 커밋 SHA 태그가 없다면 이미지가 만들어진 시각과 GitHub Actions 실행 기록을 하나씩 대조해야 합니다.

3-2. 이전 버전으로 되돌리기가 어렵습니다

Cloud Run의 이전 리비전에서 사용한 이미지가 남아 있더라도, 그 이미지가 어떤 커밋에서 만들어졌는지 바로 확인하기 어려울 수 있습니다.

이미지까지 삭제되었다면 복구에 필요한 이미지를 다시 확보해야 할 수도 있습니다.

3-3. staging과 production의 이미지가 다를 수 있습니다

현재 방식에서 staging 배포와 production 배포가 각각 이미지를 빌드한다면, 같은 코드와 Dockerfile을 사용해도 빌드 결과가 달라질 수 있습니다.

예를 들어 `requirements.txt`에 다음처럼 적혀 있다고 가정하겠습니다.

```
anthropic>=1.15
fastapi>=0.115
```

이는 특정 버전으로 고정한 것이 아니라 최소 버전을 지정한 것입니다. 두 번의 빌드 사이에 새 라이브러리 버전이 배포되면 서로 다른 버전이 설치될 수 있습니다.

Docker 베이스 이미지나 시스템 패키지도 변경될 수 있습니다. 따라서 staging에서 정상 작동했다고 해서 별도로 빌드한 production 이미지도 반드시 정상 작동한다고 보장할 수 없습니다.

4. 해결 원칙: Build Once, Deploy Many

GitHub 커밋

예: a1b2c3d

Docker 이미지 한 번 빌드

커밋 SHA 태그 + 다이제스트

Artifact Registry에 보관

Staging

검증

Production

운영

두 환경 모두 같은 다이제스트의 이미지를 사용합니다.

이 원칙을 **Build Once, Deploy Many(한 번 빌드하고 여러 환경에 배포하기)**라고 합니다.

이미지를 한 번 만들고 그 결과물을 staging과 production에서 그대로 사용하면, 환경마다 이미지를 다시 만들면서 생기는 차이를 줄일 수 있습니다.

5. 장애 대응 훈련: 어디를 확인해야 할까요?

강의에서는 실제 장애가 발생했다고 가정하고, 다음 화면을 확인합니다.

확인할 곳	확인 목적
Cloud Run → `friend-agent` → 리비전	현재 운영 중인 리비전과 이전 리비전 확인
운영 서비스의 `/version`	커밋과 리비전 정보 확인
Artifact Registry	이미지 태그, 다이제스트, 생성 시각 확인
GitHub → Actions	빌드 및 배포 기록 확인
Cloud Run → `friend-agent-staging`	staging에서 사용하는 이미지 확인

여기서 `/version`은 서비스가 커밋이나 리비전 정보를 반환하도록 구현되어 있어야 합니다.

문제를 조사할 때는 세 가지 정보를 연결합니다.

커밋 SHA → 이미지 다이제스트 → Cloud Run 리비전

이 연결이 명확하면 장애 원인을 추적하고 이전 버전으로 복구하는 작업이 쉬워집니다.

6. 실습 명령어의 목적

다음 명령어들은 환경을 점검하거나 이미지 정보를 조회하는 용도입니다. 실제 실행 시에는 `<PROJECT_ID>`, `<owner>`, `<repo>` 등을 자신의 환경에 맞게 바꾸어야 합니다.

명령어	확인하는 내용
`gcloud config list`	현재 프로젝트와 기본 설정
`gcloud run services list --region us-central1`	Cloud Run 서비스 목록
`gh variable list`	GitHub 저장소 변수
`gh api repos/{owner}/{repo}/environments --jq '.environments[].name'`	GitHub 환경 이름
`git branch -r`	원격 브랜치
`ls .github/workflows Dockerfile .dockerignore`	워크플로 및 Docker 관련 파일
`gcloud artifacts docker images list ... --include-tags`	저장된 이미지와 태그
`gcloud artifacts repositories describe ... --location us-central1`	Artifact Registry 저장소 정보

시크릿과 변수는 구별해야 합니다. 프로젝트 ID나 서비스 계정 이름 같은 설정값은 변수로 관리할 수 있지만, API 키나 인증에 필요한 민감한 값은 시크릿으로 관리해야 합니다.

또한 저장소 전체의 실제 사용량과 개별 이미지의 크기는 서로 다른 정보입니다. 저장소 크기나 요금은 콘솔과 실제 명령 결과를 통해 확인해야 합니다.

7. AI 개발자와 1인 사업가에게 중요한 이유

AI 서비스를 직접 운영하거나 고객사에 구축해 드릴 때도 이 원칙은 중요합니다.

- 서비스 신뢰성: 문제가 발생했을 때 원인을 빠르게 추적할 수 있습니다.
- 운영 비용 관리: 불필요하게 쌓이는 이미지를 정리할 수 있습니다.
- 안전한 업데이트: 새 기능을 검증한 뒤 같은 결과물을 운영 환경에 배포할 수 있습니다.
- 고객 대응: 어떤 버전에서 문제가 발생했는지 근거를 가지고 설명할 수 있습니다.
- 협업 효율: 개발자와 운영자가 동일한 이미지와 리비전을 기준으로 소통할 수 있습니다.

특히 1인 AI 사업에서는 장애 원인을 찾는 데 시간이 오래 걸릴수록 다른 업무가 중단됩니다. 버전 추적과 롤백 체계를 미리 준비하면 운영 부담을 줄일 수 있습니다.

8. 객관식 퀴즈

문제: 어제 `friend-agent:latest`로 배포했고, 오늘 다른 코드로 이미지를 만들어 같은 `latest` 태그로 올렸습니다. 어제 배포한 리비전에 대한 설명으로 옳은 것은 무엇일까요?

① 어제 리비전도 자동으로 오늘 이미지로 바뀝니다.

② 어제 리비전은 배포할 때 고정된 다이제스트의 이미지로 계속 실행되지만, 태그만 봐서는 어제와 오늘 이미지를 구별하기 어렵습니다.

③ 같은 태그로 올리면 Artifact Registry가 무조건 거부합니다.

④ 태그가 같으므로 두 이미지의 다이제스트도 같습니다.

정답 확인

9. 반드시 기억할 핵심 정리

1. 시작 상태를 확인한 뒤 빠진 항목만 준비합니다.
2. 태그는 변경될 수 있는 이름표이며, 다이제스트는 이미지 내용을 식별하는 값입니다.
3. 커밋 SHA를 이미지 태그로 사용하면 코드와 이미지의 연결을 추적하기 쉽습니다.
4. Cloud Run 리비전의 이미지 다이제스트를 확인하면 실제 배포된 이미지를 정확히 식별할 수 있습니다.
5. staging과 production이 같은 이미지를 사용하도록 Build Once, Deploy Many 원칙을 적용합니다.
6. 롤백에 필요한 이미지는 삭제하지 않도록 보존 정책을 세워야 합니다.

한 문장 요약

이번 강의는 배포 환경을 점검하고, 이미지 다이제스트를 기준으로 코드·이미지·리비전을 연결해 장애 대응과 안전한 배포를 준비하는 과정입니다.

Chapter03

/강의정리 챕터 03. 바뀌는 흐름

1. 강의 목적

이번 강의의 핵심은 Docker 이미지를 만드는 주체를 Cloud Build에서 GitHub Actions로 바꾸고, 이미지에 커밋 SHA를 붙여 코드와 배포 결과를 연결하는 것입니다.

챕터 02에서는 기존 배포 방식의 문제를 확인했습니다. 이제 실제 배포 구조를 변경합니다.

기존에는 GitHub Actions가 소스 코드를 전달하면 Cloud Build가 이미지를 만들었습니다. 앞으로는 GitHub Actions가 직접 이미지를 만들고, 커밋 SHA를 태그로 붙인 뒤 Artifact Registry에 저장합니다.

구분	기존 방식	변경 후
이미지 제작	Cloud Build	GitHub Actions 러너
이미지 저장소	`cloud-run-source-deploy`	`friend-agent`
이미지 태그	커밋을 식별하기 어려움	커밋 SHA 사용
Cloud Run 배포 입력	소스 코드	완성된 Docker 이미지
버전 추적	생성 시각과 실행 기록을 대조	이미지 태그로 커밋 추적

중요한 점은 자동 배포라는 큰 흐름은 유지하면서, 이미지를 만드는 방식과 저장 위치, 배포 입력을 바꾼다는 것입니다.

2. 중학생도 이해할 수 있는 설명

2-1. 요리 주문 방식에서 완성된 도시락 전달 방식으로

기존 방식

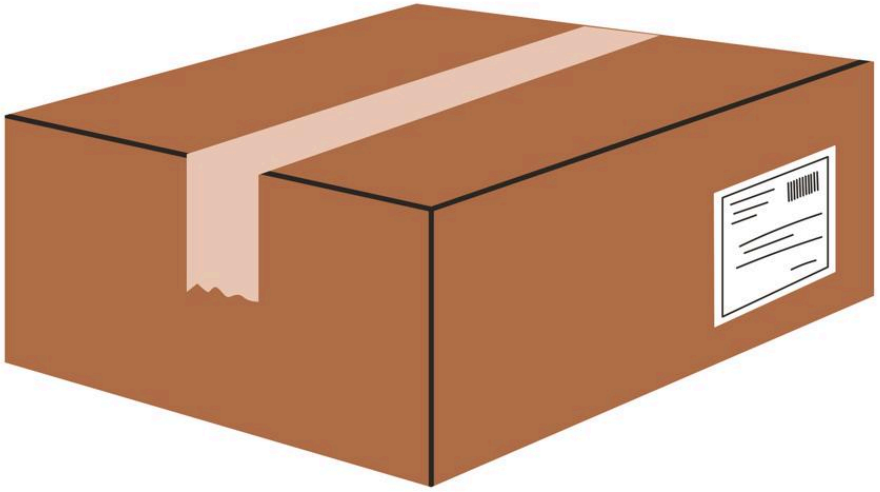


medium.com

소스 코드를 전달하고, 배포 과정에서 이미지를 제작합니다.

GitHub Actions가 레시피(소스 코드)를 보내면 Cloud Build가 요리(Docker 이미지)를 만들고 저장합니다.

변경 후



완성된 이미지를 직접 만들고, 이름을 붙여 전달합니다.

GitHub Actions가 이미지를 제작해 저장소에 올린 다음 Cloud Run에 해당 이미지의 주소를 전달합니다.

이렇게 하면 이미지가 언제, 어떤 코드로 만들어졌는지 확인하기 쉬워집니다.

2-2. 변경 후의 전체 흐름

① GitHub에 코드 머지

커밋 SHA 확정

② GitHub Actions에서 테스트

CI 통과 여부 확인

③ `docker build`

커밋 SHA로 이미지 태그 지정

④ `docker push`

Artifact Registry의 friend-agent 저장소에 업로드

⑤ Cloud Run 배포

이미지 주소를 이용해 서비스 실행

여기서 기억할 명령어는 두 가지입니다.

- `docker build`: Docker 이미지를 만듭니다.

- `docker push`: 만들어진 이미지를 원격 저장소에 올립니다.

Cloud Run 배포 단계는 소스 코드 대신 완성된 이미지 주소를 받습니다.

3. 처음 한 번 준비해야 하는 것

3-1. 이미지를 저장할 저장소 만들기

이미지를 올리려면 Artifact Registry에 저장 공간이 있어야 합니다.

예를 들어 다음 명령은 `friend-agent`라는 Docker 저장소를 만듭니다.

```
gcloud artifacts repositories create friend-agent \
  --project <PROJECT_ID> \
  --repository-format docker \
  --location us-central1
```

저장소는 완성된 Docker 이미지를 보관하는 창고입니다.

3-2. 필요한 권한만 부여하기

GitHub Actions가 이미지를 저장소에 올리려면 쓰기 권한이 필요합니다.

```
gcloud artifacts repositories add-iam-policy-binding friend-agent \
  --project <PROJECT_ID> \
  --location us-central1 \
  --member "serviceAccount:github-deployer@<PROJECT_ID>.iam.gserviceaccount.com" \
  --role roles/artifactregistry.writer
```

이 명령은 `github-deployer` 서비스 계정에 해당 저장소의 이미지 작성 권한을 부여합니다.

왜 프로젝트 전체가 아니라 저장소 하나에만 권한을 주어야 할까요?

- 프로젝트 전체 권한은 필요 이상으로 넓을 수 있습니다.
- 저장소 단위 권한은 해당 작업에 필요한 범위로 접근을 제한합니다.
- 문제가 발생하더라도 계정이 접근할 수 있는 자원을 제한할 수 있습니다.

이는 최소 권한 원칙이라고 합니다.

마지막으로 GitHub 저장소 변수에 이미지 저장소 주소를 등록합니다.

```
gh variable set AR_REPO \
  --body "us-central1-docker.pkg.dev/<PROJECT_ID>/friend-agent"
```

`AR_REPO`는 워크플로에서 이미지 주소를 조립할 때 사용하는 설정값입니다. API 키 같은 민감한 정보가 아니므로 일반 저장소 변수로 관리할 수 있습니다.

4. 워크플로 변경하기

4-1. 기존 워크플로와 달라지는 부분

강의에서는 템플릿의 `starter/workflows/deploy-image.yml`을 기존 `.github/workflows/deploy.yml`로 교체합니다.

핵심은 다음과 같습니다.

단계	역할
테스트	코드가 정상인지 확인
`setup-gcloud`	Google Cloud 명령 실행 환경 준비
`docker build`	이미지 제작
`docker push`	Artifact Registry에 업로드
`deploy-cloudrun`	완성된 이미지로 Cloud Run 배포

기존의 테스트, staging과 production 분리, production 승인 구조는 유지합니다. 이미지 제작 및 배포 부분을 변경하는 것이므로 불필요하게 다른 설정까지 바꾸지 않는 것이 중요합니다.

4-2. 커밋 SHA를 이미지 태그로 사용하기

워크플로에서 사용할 이미지 이름의 핵심은 다음과 같습니다.

```
${{ vars.AR_REPO }}/friend-agent:${{ github.sha }}
```

각 부분의 의미는 다음과 같습니다.

- `vars.AR_REPO`: 이미지 저장소 주소
- `friend-agent`: 이미지 이름
- `github.sha`: 해당 워크플로 실행과 연결된 커밋의 SHA

예를 들어 커밋 SHA가 `a1b2c3d...`라면 다음과 같은 이름을 만들 수 있습니다.

```
us-central1-docker.pkg.dev/my-project/friend-agent/friend-agent:a1b2c3d...
```

이제 Artifact Registry에서 이미지를 확인할 때 태그를 통해 GitHub의 해당 커밋으로 거슬러 올라갈 수 있습니다.

4-3. API 키를 빌드 단계에 넣으면 안 되는 이유

이미지를 만들 때 다음과 같은 명령을 추가했다고 가정하겠습니다.

```
docker build --build-arg LLM_API_KEY=... .
```

이 방식은 민감한 키가 빌드 기록이나 이미지의 빌드 정보 등에 노출될 위험을 높입니다. 빌드 인자에 전달했다고 해서 무조건 이미지에 저장되는 것은 아니지만, 안전한 기본 방식으로 사용해서는 안 됩니다.

대신 API 키는 서비스가 실행될 때 Secret Manager 등을 통해 제공하도록 설계합니다.

기억할 원칙: 비밀 키는 이미지에 넣지 않고 실행 환경에서 안전하게 제공합니다.

5. staging에서 검증하고 production으로 배포하기

이미지 기반 워크플로를 적용한 후에는 다음 순서로 확인합니다.

1. 이미지 배포 워크플로 변경을 staging 브랜치로 PR합니다.
2. CI가 통과하면 staging에 머지합니다.
3. GitHub Actions 로그에서 `docker build`, `docker push` 단계와 이미지 이름 및 다이제스트를 확인합니다.
4. Artifact Registry에서 커밋 SHA 태그가 붙은 이미지를 확인합니다.
5. staging 서비스의 `/version`에서 `commit`이 방금 배포한 커밋과 일치하는지 확인합니다.
6. staging에서 대화와 OCR 기능을 테스트합니다.
7. production 배포 PR을 진행하고 승인 절차에 따라 배포합니다.

여기서 한 가지 주의할 점이 있습니다. staging에 머지한 커밋과 main에 머지한 커밋의 SHA가 다르면, 두 환경에서 이미지 태그도 달라질 수 있습니다. main에서 이미지를 새로 빌드하면 staging에서 검증한 이미지와 production 이미지가 달라질 수 있기 때문입니다.

이번 챕터에서는 이미지 제작 방식을 변경하지만, staging에서 검증한 이미지를 그대로 production에 승격하는 방법은 다음 섹션에서 다룹니다.

6. AI 개발자와 1인 사업가에게 중요한 이유

이번 강의는 단순히 배포 명령어를 배우는 과정이 아닙니다. 서비스를 운영할 때 필요한 추적성과 보안 원칙을 익히는 과정입니다.

배운 내용	사업 운영에서의 효과
커밋 SHA 태그	장애가 발생한 코드 버전을 빠르게 확인
저장소 단위 권한	불필요한 권한을 줄여 보안 위험 완화
이미지 기반 배포	실제 배포되는 실행 결과물을 명확히 관리
시크릿 분리	API 키가 소스 코드나 이미지에 노출될 위험 감소
staging 검증	운영 배포 전 기능을 확인하는 절차 확립

예를 들어 My-Story-Maker나 AI 컨설팅용 에이전트를 고객에게 제공한다면, 기능을 추가하는 것만큼이나 어떤 버전이 고객에게 제공되고 있는지 추적할 수 있는 체계가 중요합니다.

7. 객관식 퀴즈

문제: 이미지 기반 배포로 바꾸는 PR을 검토하고 있습니다. 다음 중 머지하기 전에 반드시 수정하도록 요청해야 하는 변경은 무엇인가요?

① 이미지 태그로 `${{ github.sha }}`를 사용합니다.

② 배포 단계가 `source: .` 대신 `image: ${{ env.IMAGE }}`를 사용합니다.

③ 빌드 단계에 `--build-arg LLM_API_KEY=${{ secrets.LLM_API_KEY }}`를 추가합니다.

④ `setup-gcloud@v3` 단계를 추가합니다.

정답 확인

8. 반드시 기억할 핵심 정리

- 이미지 제작 주체를 Cloud Build에서 GitHub Actions로 변경합니다.

- Artifact Registry에 `friend-agent` 저장소를 만들고 필요한 쓰기 권한만 부여합니다.
- `docker build`로 이미지를 만들고 `docker push`로 저장소에 올립니다.
- 커밋 SHA를 태그로 사용해 코드와 이미지를 연결합니다.
- Cloud Run에는 소스 코드가 아니라 완성된 이미지 주소를 전달합니다.
- API 키는 이미지 빌드 단계에 넣지 않고 실행 환경에서 안전하게 제공합니다.
- staging 검증 후 production으로 배포하되, 검증한 이미지 자체를 승격하는 방법은 별도로 익혀야 합니다.

한 문장 요약

이번 강의는 GitHub Actions에서 커밋 SHA가 붙은 Docker 이미지를 직접 만들고 배포하도록 변경하여, 코드와 이미지의 추적성을 높이고 안전한 운영 배포의 기반을 마련하는 과정입니다.

Chapter04

/강의정리 챕터 04. 이미지 추적과 승격

1. 강의 목적

이번 강의의 핵심은 어떤 커밋으로 만든 Docker 이미지가 실제 운영 환경에서 실행되는지 추적하고, staging에서 검증한 바로 그 이미지를 production에 그대로 배포하는 것입니다.

챕터 03에서는 GitHub Actions가 이미지를 직접 만들고 커밋 SHA를 태그로 붙이도록 변경했습니다. 이번 챕터에서는 한 단계 더 나아가 다음 세 가지 문제를 해결합니다.

1. 현재 production에서 실행 중인 코드는 어느 커밋인가?
2. 문제가 발생하면 정상적으로 동작하던 이전 이미지로 되돌릴 수 있는가?
3. staging에서 검증한 이미지와 production의 이미지가 정말 같은가?

이를 위해 두 가지 작업을 합니다.

- 연결표 작성: 커밋, 이미지, 다이제스트, Cloud Run 리비전, `/version` 응답을 연결합니다.
- 이미지 승격(Image Promotion): staging에서 검증한 이미지를 다시 빌드하지 않고 production에 그대로 배포합니다.

2. 중학생도 이해할 수 있는 설명

2-1. 제품의 제조번호부터 실제 판매처까지 추적하기

어떤 제품에서 문제가 발생했다고 가정하겠습니다. 제품에 제조번호가 없다면 언제, 어떤 재료로 만들었는지 알아내기 어렵습니다.

소프트웨어에서도 마찬가지입니다.

① Git 커밋 SHA

어떤 코드로 만들었는가?

② Docker 이미지 태그

어떤 커밋으로 만든 이미지인가?

③ 이미지 다이제스트

이미지의 정확한 식별값은 무엇인가?

④ Cloud Run 리비전

어떤 이미지가 실제로 배포되었는가?

⑤ `/version` 응답

서비스가 보고하는 커밋과 리비전은 무엇인가?

이 다섯 단계를 연결하면 운영 중인 서비스를 조사할 때 추측에 의존하지 않고 실제 기록을 따라갈 수 있습니다.

2-2. 연결표는 소프트웨어의 이력 추적표입니다

강의에서는 다음과 같은 표를 staging과 production에 대해 작성합니다.

환경	커밋 SHA	이미지 태그	다이제스트	리비전 이름	`/version`
staging	확인할 값	확인할 값	확인할 값	확인할 값	확인할 값
production	확인할 값	확인할 값	확인할 값	확인할 값	확인할 값
production (승격 후)	확인할 값	확인할 값	확인할 값	확인할 값	확인할 값

실제 값은 각자의 Google Cloud 프로젝트와 GitHub 저장소에서 조회해야 합니다. 강의 자료에는 사용자의 실제 배포 결과가 포함되어 있지 않으므로 임의의 값을 채우면 안 됩니다.

커밋 SHA와 다이제스트는 길기 때문에 비교를 위한 메모에는 앞 12자리 정도를 적어도 됩니다. 다만 실제 배포 명령에서는 다이제스트 전체를 사용해야 합니다.

3. 연결표 채우기: 실제 배포 버전 추적하기

3-1. 어디에서 무엇을 확인할까요?

확인할 정보	확인 방법
main의 마지막 커밋	`git log -1 --format=%H origin/main`
이미지 태그와 다이제스트	Artifact Registry 이미지 목록
Cloud Run 리비전	`gcloud run revisions list`
리비전이 사용하는 이미지	`gcloud run revisions describe`
서비스가 보고하는 버전	서비스 주소 뒤에 `/version` 추가

예를 들어 Cloud Run 리비전 상세에 다음과 같은 이미지 주소가 있다고 하겠습니다.

`.../friend-agent@sha256:0d01a69...`

이 주소의 다이제스트를 Artifact Registry에서 찾으면 해당 이미지에 붙은 커밋 SHA 태그를 확인할 수 있습니다.

이렇게 리비전에서 이미지로, 이미지에서 커밋으로 거슬러 올라가는 것이 핵심입니다.

3-2. `/version`과 이미지 태그가 다르면?

예를 들어 다음과 같은 상황입니다.

- `/version`의 `commit`: `a1b2c3d`
- 리비전 이미지의 태그: `e4f5g6h`

이 경우 곧바로 어느 한쪽이 틀렸다고 단정하면 안 됩니다. 다음 순서로 확인합니다.

1. Cloud Run 리비전 상세에서 실제 이미지 다이제스트를 확인합니다.
2. Artifact Registry에서 그 다이제스트에 대응하는 이미지 태그를 찾습니다.
3. `/version`의 `commit`이 해당 이미지 제작 당시의 커밋을 가리키는지 확인합니다.
4. 배포 워크플로에서 이미지와 `COMMIT_SHA` 환경변수가 같은 버전을 가리키도록 전달되었는지 확인합니다.

이미지만 새 버전으로 바꾸고 `COMMIT_SHA` 환경변수를 갱신하지 않았다면, `/version`은 실제 이미지와 다른 커밋을 표시할 수 있습니다.

4. 이미지 승격(Image Promotion)

4-1. 왜 이미지 승격이 필요할까요?

챕터 03의 방식으로 staging과 production에서 각각 이미지를 만들면, 같은 코드라도 이미지가 서로 달라질 수 있습니다.

예를 들어 staging에서 월요일에 이미지를 만들고 수요일에 production 이미지를 다시 빌드했다면, 그사이 라이브러리 버전이나 시스템 패키지가 변경되었을 수 있습니다.

이미지 승격은 이 문제를 해결합니다.

GitHub Actions에서 이미지 한 번 빌드

커밋 SHA 태그와 다이제스트 생성

Staging에서 테스트

대화, OCR 등 기능 검증

동일한 다이제스트로 Production 배포

이미지를 다시 빌드하지 않음

이것이 챕터 02에서 배운 Build Once, Deploy Many를 실제 배포 구조에 적용하는 방법입니다.

4-2. 왜 태그 대신 다이제스트를 사용하나요?

이미지를 승격할 때는 다음과 같은 주소를 사용합니다.

`us-central1-docker.pkg.dev/my-project/friend-agent/friend-agent@sha256:...`

태그는 다른 이미지로 옮겨 붙일 수 있지만, 다이제스트는 특정 이미지 내용을 식별합니다.

따라서 staging에서 검증한 다이제스트를 production 배포에 그대로 사용하면 두 환경이 동일한 이미지 내용을 사용하도록 할 수 있습니다.

주의: 다이제스트는 이미지 자체를 지정하지만, 환경변수나 시크릿, 서비스 계정 같은 실행 설정까지 자동으로 동일하게 만들어 주지는 않습니다.

4-3. Production 승격 시 커밋 정보도 함께 변경합니다

강의 자료에서는 이미지를 승격할 때 `COMMIT_SHA`도 staging에서 사용하던 값으로 변경합니다.

```
gcloud run deploy friend-agent \
  --region us-central1 \
  --image <staging에서_확인한_다이제스트_주소> \
  --update-env-vars COMMIT_SHA=<staging의_커밋_SHA>
```

여기서 중요한 옵션은 `--update-env-vars`입니다.

- `-update-env-vars`: 지정한 환경변수를 갱신하면서 다른 기존 환경변수를 유지합니다.
- `-set-env-vars`: 지정한 환경변수 집합으로 설정을 교체할 수 있어, 기존 설정을 의도치 않게 잃을 위험이 있습니다.

이 명령은 production 서비스를 실제로 변경하므로, staging 검증과 이미지 주소 확인을 마친 뒤 실행해야 합니다. 또한 기존 배포 워크플로와 별개로 직접 배포하는 방식이라는 점도 알아 두셔야 합니다.

5. 이미지 승격을 자동화하기

매번 사람이 다이제스트를 찾아 직접 배포하면 번거롭습니다. 따라서 강의에서는 수동으로 검증한 절차를 GitHub Actions 워크플로로 옮깁니다.

변경할 곳은 세 군데입니다.

변경 대상	변경 내용
① 이미지 제작 단계	staging 배포에서만 실행
② staging 이미지 조회 단계	main 배포에서 staging 리비전의 이미지 다이제스트와 커밋을 조회
③ 배포 단계	조회한 이미지와 커밋을 사용

main 배포에서 새 이미지를 만들지 않고 staging 이미지를 가져오는 것이 핵심입니다.

워크플로에서는 `$GITHUB_ENV`에 저장한 값을 같은 작업의 다음 단계에서 `${{ env.IMAGE }}`, `${{ env.COMMIT }}`으로 사용할 수 있습니다.

다만 staging 이미지 조회 단계에서는 실제 트래픽을 받는 리비전의 이미지와 `COMMIT_SHA`를 올바르게 가져오는지 확인해야 합니다. 또한 production 배포 승인 전에 staging에서 검증한 버전이 여전히 승격 대상인지 확인해야 합니다.

6. 로컬에서 만든 이미지를 배포하지 않는 이유

강의에서는 배포용 이미지를 GitHub Actions에서만 만들도록 작업 규칙에 추가합니다.

그 이유는 두 가지입니다.

이유 1. CPU 아키텍처가 다를 수 있습니다

Docker 이미지는 CPU가 실행할 수 있는 명령 형식에 맞춰 만들어집니다.

- `linux/amd64`: 일반적인 Intel·AMD 서버용 아키텍처
- `linux/arm64`: ARM 기반 아키텍처

예를 들어 Apple Silicon Mac에서 기본 설정으로 만든 이미지가 `arm64`라면, `linux/amd64` 이미지를 요구하는 Cloud Run 배포에서 문제가 발생할 수 있습니다.

필요하다면 `--platform linux/amd64`로 빌드할 수 있지만, 이것만으로 모든 문제가 해결되는 것은 아닙니다.

이유 2. 어떤 코드로 만들었는지 기록이 끊깁니다

로컬에서 만든 이미지는 GitHub에 커밋되지 않은 파일이나 실험용 수정 사항을 포함할 수 있습니다.

그 경우 다음 연결이 불분명해집니다.

- GitHub 커밋
- PR 검토와 CI 테스트
- Docker 이미지
- Cloud Run 리비전

반면 GitHub Actions에서 커밋된 코드를 기준으로 이미지를 만들면 빌드 이력을 추적하기 쉽고, 동일한 소스 버전을 확인할 수 있습니다.

따라서 저장소 루트의 `AGENTS.md`에 다음 규칙을 추가합니다.

> 배포용 이미지는 GitHub Actions에서만 만든다. 로컬 환경에서 만든 이미지를 Artifact Registry에 올리거나 배포하지 않는다.

이 규칙은 코딩 에이전트가 편의를 위해 검증 절차를 생략하는 일을 방지하는 데도 도움이 됩니다.

7. AI 개발자와 1인 사업가에게 중요한 이유

AI 에이전트 서비스를 직접 운영하거나 고객사에 제공할 때도 이번 강의의 원칙은 그대로 적용됩니다.

실무 상황	학습 내용의 활용
고객이 오류를 신고함	`/version`에서 커밋을 확인하고 변경 이력 추적
업데이트 후 장애 발생	이전 이미지와 리비전을 확인해 롤백 검토
테스트는 성공했지만 운영에서 오류 발생	staging과 production의 다이제스트 비교
자동 배포 과정에서 실수 발생	워크플로와 권한 설정을 기준으로 원인 조사
여러 프로젝트를 운영	프로젝트별 이미지와 배포 버전 관리

1인 AI 사업에서는 개발자와 운영 담당자가 분리되어 있지 않을 수 있습니다. 그러므로 자동 배포를 구축하는 것뿐 아니라 문제가 발생했을 때 무엇이 실행 중인지 증명하고 안전하게 복구할 수 있는 구조를 마련하는 것이 중요합니다.

8. 객관식 퀴즈

문제 1. staging 이미지를 production으로 승격할 때 태그 대신 `@sha256:` 다이제스트 주소를 사용하는 가장 중요한 이유는 무엇일까요?

① 태그 주소로는 Cloud Run에 배포할 수 없기 때문입니다.

② 다이제스트는 이미지 내용을 식별하므로 staging에서 확인한 이미지 하나를 정확히 지정할 수 있기 때문입니다.

③ 다이제스트를 사용하면 환경변수가 자동으로 staging 값으로 바뀌기 때문입니다.

④ 다이제스트를 사용하면 다음 main 머지 때 이미지를 다시 만들지 않기 때문입니다.

정답 확인

문제 2. 로컬에서 만든 이미지를 배포하지 않도록 규칙을 정하는 이유로 가장 알맞은 것은 무엇일까요?

① 로컬 Docker에서는 이미지에 태그를 붙일 수 없기 때문입니다.

② 로컬 이미지는 Artifact Registry에 업로드할 수 없기 때문입니다.

③ CPU 아키텍처가 맞지 않을 수 있고, 어떤 코드로 만들었는지 검증 가능한 기록이 끊길 수 있기 때문입니다.

④ 로컬에서 만든 이미지는 항상 더 크기 때문입니다.

정답 확인

9. 반드시 기억할 핵심 정리

- 연결표: 커밋 SHA → 이미지 태그 → 다이제스트 → Cloud Run 리비전 → `/version`을 연결합니다.
- 이미지 승격: staging에서 검증한 이미지를 다시 빌드하지 않고 production에 배포합니다.
- 다이제스트: 정확히 어떤 이미지인지 지정할 때 사용합니다.
- 환경변수: 이미지 승격 시 `COMMIT_SHA`도 함께 갱신해야 실제 이미지와 버전 정보가 일치합니다.
- 워크플로 자동화: staging에서 이미지를 만들고, main에서는 staging 이미지를 조회해 production으로 승격합니다.
- 로컬 빌드 금지: 배포용 이미지는 GitHub Actions에서만 만들어 코드와 배포 이력을 연결합니다.
- 장애 대응: 실제 실행 중인 이미지와 커밋을 확인한 뒤 롤백 여부를 판단합니다.

한 문장 요약

이번 강의는 커밋부터 실행 중인 리비전까지 추적하는 연결표를 만들고, staging에서 검증한 동일한 Docker 이미지를 production에 승격하도록 자동화하는 과정입니다.

Chapter05

/강의정리 챕터05. 정리 정책 정하기

1. 학습 목적

Docker 이미지와 배포 환경을 관리할 때는 이미지를 만드는 것만큼 불필요한 이미지를 정리하고, 문제가 생겼을 때 되돌아갈 이미지를 남겨 두는 것도 중요합니다.

이번 수업의 핵심은 다음과 같습니다.

- Artifact Registry의 이미지 정리 정책 이해하기
- 삭제 정책과 유지 정책을 함께 설정하기
- 실제 삭제 전에 드라이런(Dry Run)으로 결과 확인하기
- 저장 공간을 절약하면서 롤백에 필요한 이미지는 보존하기

2. 중학생도 이해할 수 있는 설명

2-1. 이미지가 계속 쌓이면 어떻게 될까요?

프로그램을 배포할 때마다 실행에 필요한 이미지가 저장소에 쌓입니다. 마치 컴퓨터에 파일을 계속 저장하는 것과 같습니다.

friend-agent 저장소

직접 만든 애플리케이션 이미지가 쌓이는 곳

cloud-run-source-deploy 저장소

소스 배포 과정에서 생성된 이미지가 남아 있는 곳

강의 자료에 따르면 무료 저장 공간은 0.5GB이므로, 오래된 이미지가 쌓이면 저장 공간을 초과할 수 있습니다.

이 문제를 해결하기 위해 저장소에 정리 정책을 설정합니다. 일정 조건에 맞는 이미지를 자동으로 삭제하거나, 필요한 이미지를 남기도록 규칙을 정하는 것입니다.

2-2. 삭제 정책과 유지 정책의 차이

구분	의미	이번 수업의 예
삭제 정책(Delete)	조건에 맞는 이미지를 삭제 대상으로 지정	7일보다 오래된 이미지
유지 정책(Keep)	조건에 맞는 이미지를 보존	최근 이미지 10개
드라이런(Dry Run)	실제 삭제 없이 삭제 대상을 미리 확인	정책을 시험 실행

핵심은 삭제 정책과 유지 정책을 함께 설정하면 두 정책에 동시에 해당하는 이미지는 유지된다는 점입니다.

2-3. 이번 수업의 정리 규칙

규칙 1. 7일보다 오래된 이미지는 삭제 대상

오래되어 불필요해진 이미지를 정리합니다.

규칙 2. 최근 이미지 10개는 유지

이미지가 오래되었더라도 최근 10개에 해당하면 보존합니다.

규칙 3. 처음에는 드라이런으로 확인

무엇이 삭제될지 먼저 확인하고, 결과가 예상과 일치할 때 실제 적용합니다.

최근 10개를 남기는 이유는 롤백(rollback) 때문입니다. 현재 배포에 문제가 생기면 이전에 사용하던 이미지로 되돌아가야 하는데, 그 이미지가 이미 삭제되었다면 롤백할 수 없기 때문입니다.

3. 비유로 이해하기

이미지 저장소를 문서 보관실이라고 생각해 보겠습니다.

- 삭제 정책: 7일이 지난 문서는 폐기하세요.
- 유지 정책: 단, 최근 문서 10개는 무조건 보관하세요.
- 드라이런: 실제로 버리기 전에 폐기 목록을 먼저 작성하세요.
- 롤백: 문제가 생겼을 때 이전 문서를 다시 꺼내 보는 것입니다.

오래된 문서라고 무조건 버리면 안 됩니다. 현재 업무에 문제가 생겼을 때 과거 기록이 필요할 수 있기 때문입니다.

4. 실제 프로젝트에 적용하기

4-1. 정책 파일 만들기

강의에서 제시한 `docs/cleanup-policy.json`의 내용입니다.

```
[
  {
    "name": "delete-older-than-7d",
    "action": {"type": "Delete"},
    "condition": {
      "tagState": "any",
      "olderThan": "7d"
    }
  },
  {
    "name": "keep-recent-10",
    "action": {"type": "Keep"},
    "mostRecentVersions": {"keepCount": 10}
  }
]
```

4-2. 두 저장소에 드라이런 적용하기

```
gcloud artifacts repositories set-cleanup-policies friend-agent \
  --project <PROJECT_ID> \
  --location us-central1 \
  --policy docs/cleanup-policy.json \
  --dry-run
```

```
gcloud artifacts repositories set-cleanup-policies cloud-run-source-deploy \
  --project <PROJECT_ID> \
  --location us-central1 \
  --policy docs/cleanup-policy.json \
  --dry-run
```

`<PROJECT_ID>`는 실제 Google Cloud 프로젝트 ID로 바꾸어야 합니다.

주의: 이번 실습에서는 `--dry-run`을 사용합니다. 실제 삭제를 수행하는 `--no-dry-run`은 결과를 확인한 다음에 사용합니다.

4-3. 실행 전 확인할 항목

- [ ] 저장소 이름이 `friend-agent`, `cloud-run-source-deploy`로 정확한가?
- [ ] 정책 파일에 삭제 정책과 유지 정책이 모두 있는가?
- [ ] 두 명령에 `-dry-run`이 들어 있는가?
- [ ] 드라이런 결과에서 현재 production 이미지가 삭제 대상인지 확인했는가?
- [ ] 바로 이전 배포 이미지와 롤백 대상 이미지가 보존되는가?

강의 자료에서는 정책 변경이 반영되는 데 하루 정도 걸릴 수 있으며, 드라이런 결과는 이후 감사 로그에서 확인할 수 있다고 설명합니다(2026년 10월 기준).

5. 퀴즈 풀이

문제: 이미지가 15개이고, 그중 6개가 7일보다 오래되었습니다. 최근 10개를 유지하는 정책을 적용하면 몇 개가 삭제될까요?

정답을 선택해 보세요.

1. 0개

2. 5개

3. 6개

4. 9개

정답 확인

6. AI 개발자가 알아야 하는 이유

AI 서비스를 개발하고 운영하면 코드뿐 아니라 실행 이미지, 모델 관련 파일, 로그 등 다양한 자원이 누적될 수 있습니다.

정리 정책을 제대로 설계하지 않으면 다음과 같은 문제가 생깁니다.

- 저장 공간이 불필요하게 늘어납니다.
- 현재 서비스에 필요한 이미지가 삭제될 수 있습니다.
- 장애가 발생했을 때 이전 버전으로 복구하기 어려워집니다.
- 운영 비용과 장애 대응 위험이 함께 증가합니다.

따라서 자동 정리와 안전한 복구를 함께 설계하는 것이 안정적인 AI 서비스 운영의 기본입니다.

7. 학습 및 창업 목표와 연결

앞으로 준비하실 1인 AI 컨설팅 사업이나 AI 에이전트 서비스에서도 동일한 원칙이 적용됩니다.

- 서비스를 배포하는 절차만 만들지 말고, 이미지 보존과 정리 기준도 문서화합니다.
- 자동 삭제를 바로 활성화하지 말고, 드라이런으로 먼저 검증합니다.
- 장애 발생 시 어느 버전으로 돌아갈 수 있는지 확인합니다.

이는 단순한 개발 기술이 아니라, 고객에게 안정적인 서비스를 제공하기 위한 운영 관리 능력입니다.

8. 핵심 정리

- 삭제 정책은 불필요한 이미지를 정리합니다.
- 유지 정책은 롤백에 필요한 이미지를 보존합니다.
- 이번 정책은 7일보다 오래된 이미지를 삭제 대상으로 삼되, 최근 10개는 유지합니다.
- 처음에는 `-dry-run`으로 삭제 대상을 검토합니다.
- `friend-agent`에는 staging과 production 이미지가 함께 쌓이므로 두 환경의 롤백 필요성을 함께 고려해야 합니다.

반드시 기억할 것

> 이미지 정리의 목적은 오래된 이미지를 무조건 삭제하는 것이 아니라, 안전한 롤백에 필요한 이미지는 남기면서 불필요한 저장 공간을 줄이는 것입니다.

한 문장 요약: 정리 정책은 삭제 정책과 유지 정책을 함께 설정하고, 드라이런으로 결과를 확인한 뒤 실제 적용해야 안전합니다.

Chapter06

/강의정리 챕터06. 고장 난 Dockerfile 고치기

1. 학습 목적

이번 수업은 Dockerfile의 일부를 의도적으로 변경한 뒤, 프로그램의 증상만 보고 원인을 추론하고 정상 상태로 복구하는 훈련입니다.

Dockerfile은 단순히 이미지를 만드는 설명서가 아닙니다. 한 줄이 달라져도 이미지에 포함되는 파일, 프로그램을 실행하는 사용자, 네트워크 포트 등이 달라질 수 있습니다.

이번에 확인할 문제는 네 가지입니다.

- A. OCR 기능만 실패하는 문제
- B. 이미지에 불필요하거나 민감한 파일이 포함되는 문제
- C. 프로그램이 실행되는 사용자가 바뀌는 문제
- D. 컨테이너는 실행되지만 브라우저에서 접속할 수 없는 문제

2. 중학생도 이해할 수 있는 설명

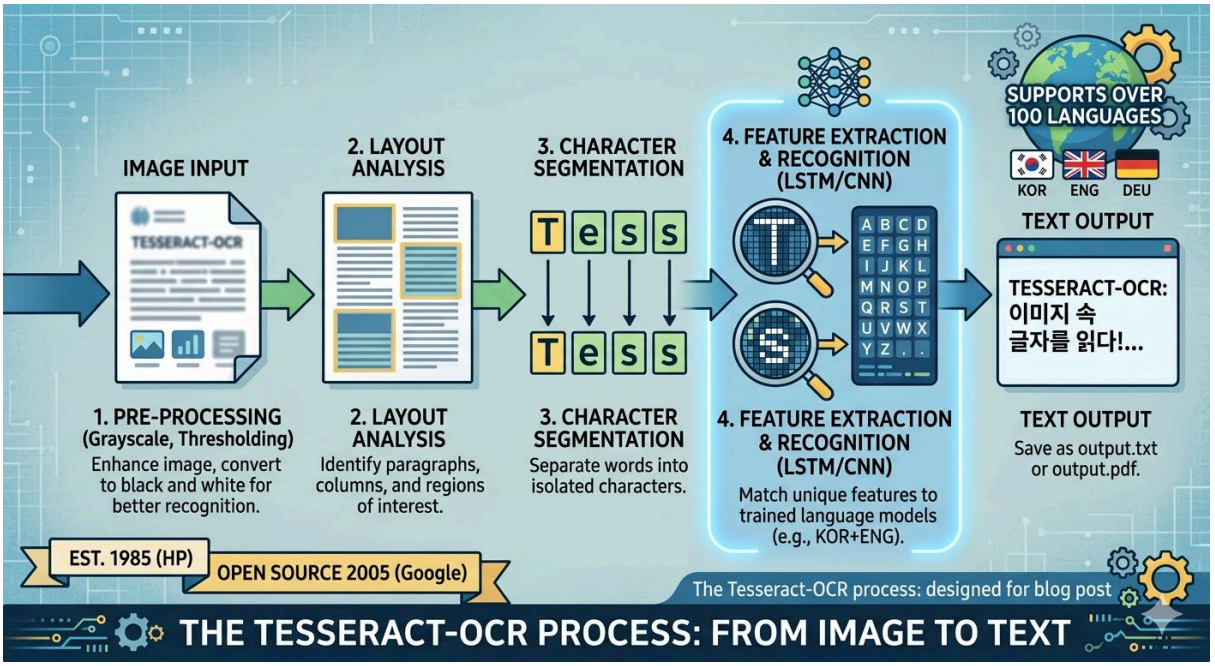
Dockerfile을 음식을 만드는 조리법이라고 생각해 보겠습니다.

- 필요한 재료를 넣는 것 → 이미지에 필요한 파일을 포함하는 것
- 요리하는 사람을 정하는 것 → 컨테이너의 실행 사용자 지정
- 음식을 내놓는 창구를 정하는 것 → 서비스가 요청을 기다리는 포트 설정

조리법의 한 줄이 잘못되면 음식은 만들어져도 특정 재료가 빠지거나, 주방 운영 방식이 달라지거나, 손님이 음식을 받을 창구를 찾지 못할 수 있습니다.

마찬가지로 이미지가 정상적으로 빌드되었다고 해서 서비스까지 정상이라는 뜻은 아닙니다.

3. 네 가지 고장 유형



문제 A. OCR만 실패하는 이미지

특정 기능 장애

대화 기능은 정상인데 이미지에서 글자를 읽는 OCR 기능만 실패합니다. Dockerfile에서 OCR에 필요한 프로그램이 이미지에 포함되지 않았을 가능성을 의심합니다.

확인 방법: OCR 기능을 켜고 `schedule.png`를 전송합니다.

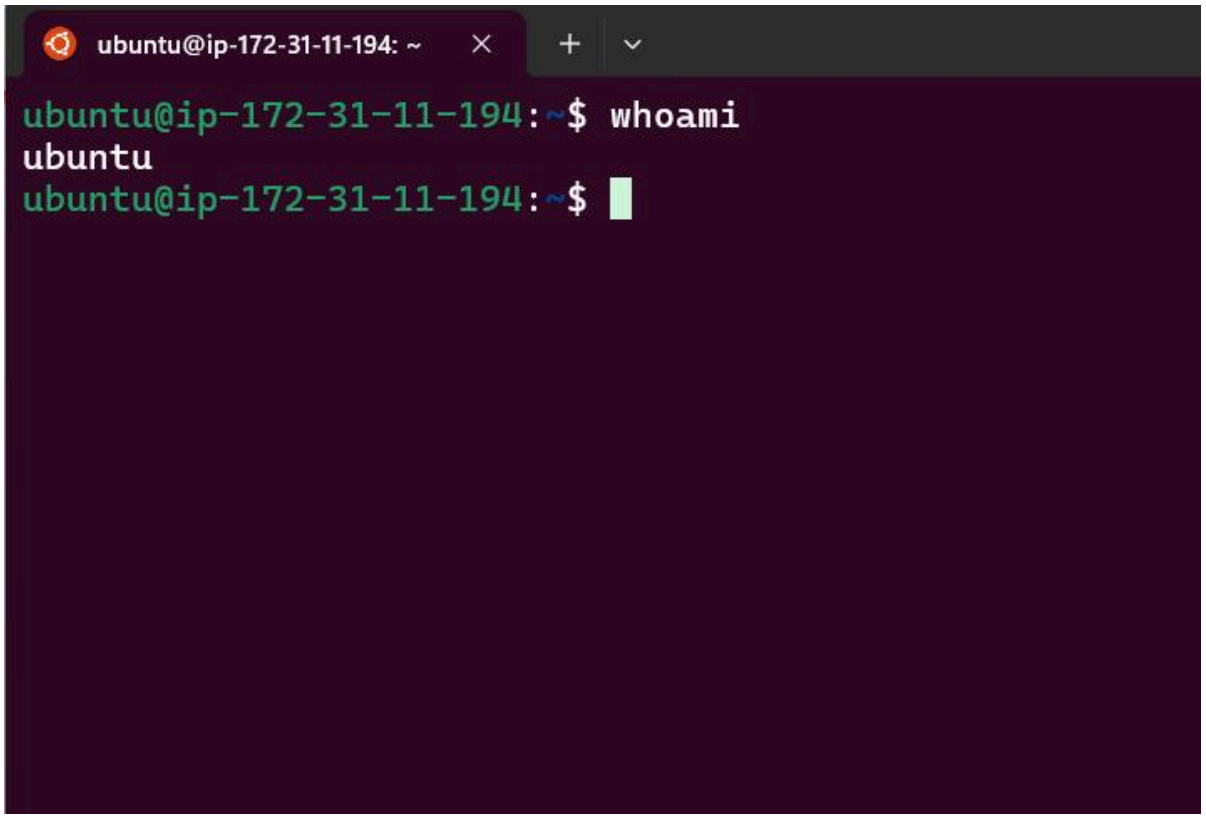
```
~ via v23.9.0
✅ docker exec my-nginx ls -la
total 20
drwxr-xr-x.   1 root root   18 Mar 28 08:58 .
drwxr-xr-x.   1 root root   18 Mar 28 08:58 ..
-rwxr-xr-x.   1 root root    0 Mar 28 08:58 .dockerenv
lrwxrwxrwx.   1 root root    7 Jul 22  2024 bin -> usr/bin
drwxr-xr-x.   1 root root    0 Mar 29  2024 boot
drwxr-xr-x.   5 root root  340 Mar 28 08:58 dev
drwxr-xr-x.   1 root root   54 Jul 23  2024 docker-entrypoint.d
-rwxr-xr-x.   1 root root 1620 Jul 23  2024 docker-entrypoint.sh
drwxr-xr-x.   1 root root   10 Mar 28 08:58 etc
drwxr-xr-x.   1 root root    0 Mar 29  2024 home
lrwxrwxrwx.   1 root root    7 Jul 22  2024 lib -> usr/lib
lrwxrwxrwx.   1 root root    9 Jul 22  2024 lib64 -> usr/lib64
drwxr-xr-x.   1 root root    0 Jul 22  2024 media
drwxr-xr-x.   1 root root    0 Jul 22  2024 mnt
drwxr-xr-x.   1 root root    0 Jul 22  2024 opt
dr-xr-xr-x. 628 root root    0 Mar 28 08:58 proc
drwx------.   1 root root   30 Jul 22  2024 root
drwxr-xr-x.   1 root root   18 Mar 28 08:58 run
lrwxrwxrwx.   1 root root    8 Jul 22  2024 sbin -> usr/sbin
drwxr-xr-x.   1 root root    0 Jul 22  2024 srv
dr-xr-xr-x.  13 root root    0 Mar 28 07:37 sys
drwxrwxrwt.   1 root root    0 Jul 22  2024 tmp
drwxr-xr-x.   1 root root   40 Jul 22  2024 usr
drwxr-xr-x.   1 root root   10 Jul 22  2024 var
~ via v23.9.0
✅
```

문제 B. 이미지 안에 들어간 파일

정보 노출 위험

`.dockerignore`는 Docker 이미지에 복사하지 않을 파일을 지정하는 데 사용됩니다. 설정이 잘못되면 `.env` 같은 민감한 파일이 이미지에 포함될 수 있습니다.

확인 방법: `docker run --rm friend-agent:broken-b ls -a /app`로 파일 목록을 확인합니다.

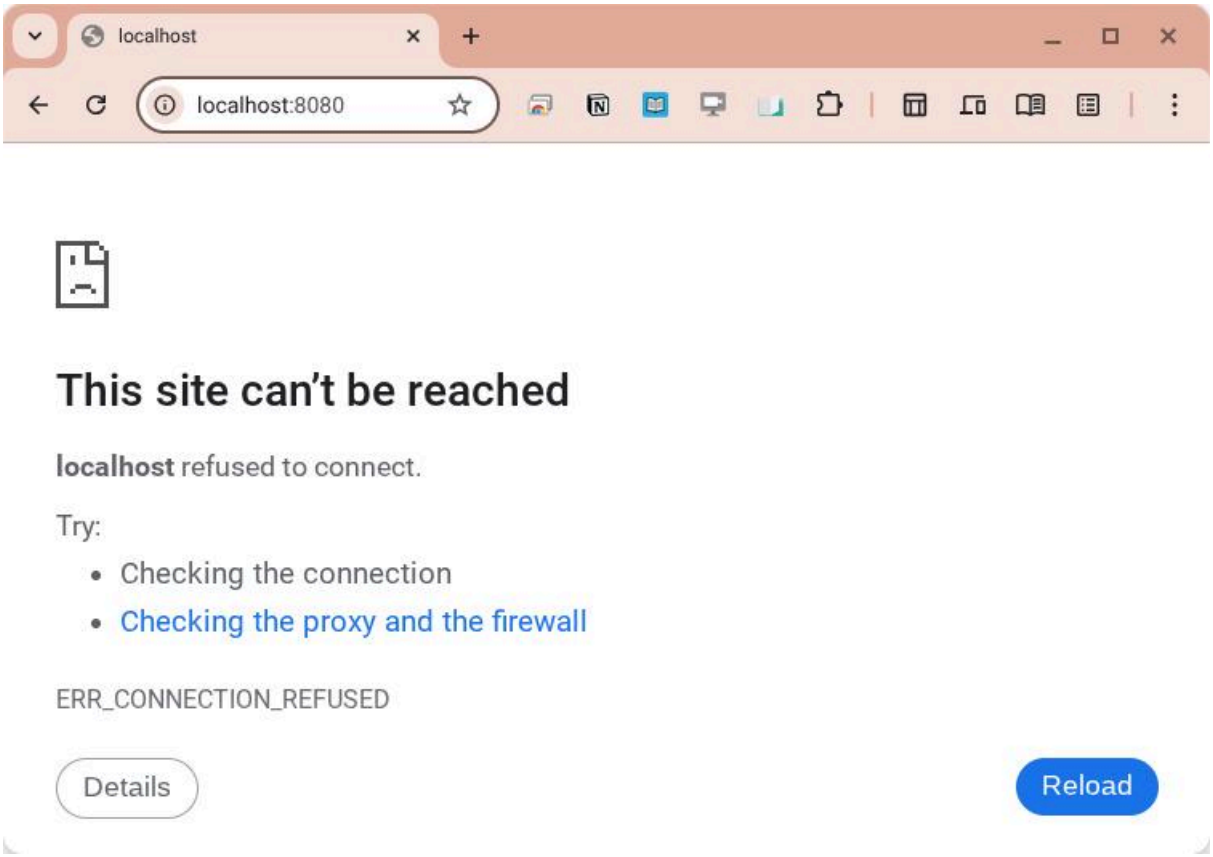


문제 C. 실행하는 사용자가 바뀐 이미지

겉으로 드러나지 않는 차이

서비스는 정상적으로 동작하더라도, 원래 의도한 사용자 대신 다른 사용자로 실행될 수 있습니다. 실행 권한과 보안에 영향을 줄 수 있습니다.

확인 방법: `docker run --rm friend-agent:broken-c whoami`로 실행 사용자를 확인합니다.



문제 D. 접속되지 않는 컨테이너

서비스 접속 장애

이미지는 만들어지고 컨테이너도 실행되지만 브라우저에서 접속되지 않습니다. 프로그램이 실제로 요청을 기다리는 포트와 컨테이너 설정이 맞는지 확인해야 합니다.

확인 방법: `http://localhost:8080`에 접속하고 컨테이너 로그의 첫 줄을 확인합니다.

위 내용은 강의에서 제시한 증상에 따른 원인 후보입니다. 실제로 어느 줄이 바뀌었는지는 해당 파일과 로그를 확인해야 확정할 수 있습니다.

4. 실습 진행 순서

이번 수업에서 중요한 것은 단순히 정답을 맞히는 것이 아니라, 증상 → 원인 가설 → 확인 → 복구 → 재검증의 순서를 익히는 것입니다.

1단계. 변경된 파일 적용

실습용 Dockerfile 또는 `.dockerignore`를 루트에 복사합니다.

2단계. 이미지를 만들고 증상 확인

로컬 컨테이너나 staging 환경에서 실제 결과를 관찰합니다.

3단계. 원인 추론 후 비교

먼저 증상과 로그로 가설을 세운 뒤 원본 파일과 비교합니다.

4단계. 원상 복구 및 재확인

원래 파일로 되돌리고 정상 동작을 확인합니다.

실습에서 주의할 점

- `git diff`를 먼저 열어보면 무엇이 바뀌었는지 바로 알 수 있으므로, 강의에서는 증상과 기록을 먼저 보고 원인을 추측한 뒤 마지막에 비교하도록 안내합니다.
- 문제 B에서 만든 이미지는 저장소에 올리지 않습니다. 이미지에 포함된 파일은 이미지를 받은 사람도 꺼내 볼 수 있으므로 확인 후 이미지를 삭제합니다.
- 실습이 끝나면 `git status`로 `Dockerfile`과 `.dockerignore`가 원래 상태인지 확인하고 연습용 이미지를 정리합니다.

5. AI 코딩 에이전트 활용법

코딩 에이전트에게는 처음부터 고치라고 하기보다, 먼저 원인을 분석하도록 요청하는 것이 좋습니다.

루트의 Dockerfile 또는 .dockerignore로 만든 이미지에서 다음 증상이 나타납니다.

[관찰한 증상과 로그를 여기에 입력합니다.]

아직 파일을 수정하지 마세요.

1. 증상과 로그를 근거로 가능한 원인을 설명해 주세요.
2. Dockerfile 또는 .dockerignore의 어느 설정이 관련되어 있을지 가설을 세워 주세요.
3. 각 가설을 확인할 수 있는 명령을 제안해 주세요.
4. 원본 파일과 비교하기 전에 원인을 먼저 추론해 주세요.
5. 실제 파일 수정이나 배포 명령은 실행하지 마세요.

분석이 끝나면 원인과 근거를 먼저 보고해 주세요.

이렇게 하면 에이전트가 파일을 무작정 고치는 대신, 관찰한 증거를 바탕으로 문제를 진단하는 과정을 배울 수 있습니다.

6. 주관식 퀴즈

질문: 네 문제 가운데 GitHub Actions에서 만든 이미지로 배포했다면 드러나지 않았을 문제와, 서비스가 정상으로 보여서 그냥 지나쳤을 문제는 각각 무엇일까요?

다음은 강의 내용에 근거한 답변 예시입니다.

① GitHub Actions 배포 이미지로는 드러나지 않았을 문제

문제 B입니다. 이 문제는 `.dockerignore`가 변경된 경우로, 강의에서는 로컬에서 이미지를 만들어 내부 파일 목록을 확인하도록 안내합니다. GitHub Actions에서 정상적인 설정으로 별도의 이미지를 빌드했다면, 잘못된 로컬 `.dockerignore` 설정은 그 배포 이미지에 반영되지 않을 수 있습니다.

② 서비스가 정상으로 보여서 그냥 지나쳤을 문제

문제 C입니다. 실행 사용자가 바뀌어도 서비스 자체는 정상적으로 동작할 수 있습니다. 따라서 화면에서 기능만 확인하면 놓칠 수 있으며, `whoami` 명령 등으로 실행 사용자를 별도로 확인해야 합니다.

7. 선택 과제와 실무 연결

과제	핵심 학습	실무적 의미
1. 손으로 롤백 훈련	이전 이미지로 복구하고 다시 최신 버전으로 전환	장애 복구 능력

과제	핵심 학습	실무적 의미
2. 태그 덮어쓰기 막기	변경 불가능한 태그 설정 실험	배포 이미지 식별의 신뢰성
3. slim 이미지와 전체 이미지 비교	베이스 이미지 크기 비교	저장 공간과 배포 시간 최적화
4. 배포 기록 자동화	커밋·리비전·다이제스트 기록	배포 이력 추적과 운영 자동화

특히 과제 1과 4는 앞서 배운 내용과 직접 연결됩니다.

- 롤백: 이전 이미지가 저장소에 남아 있어야 하며, `COMMIT_SHA`도 해당 이미지에 맞춰야 합니다.
- 배포 기록: 커밋 SHA, Cloud Run 리비전, 이미지 다이제스트를 연결해 두면 어떤 코드가 실제 서비스에서 실행되는지 추적할 수 있습니다.
- 태그 관리: 커밋 SHA 태그가 다른 이미지로 이동하지 않도록 관리할 수 있습니다. 다만 태그 변경 금지 설정은 이미지 삭제를 제한할 수 있어 정리 정책과 함께 검토해야 합니다.

8. AI 개발자가 반드시 기억할 것

1. 빌드 성공과 서비스 정상 동작은 다릅니다. 이미지가 만들어져도 기능, 파일 구성, 실행 사용자, 포트 설정에 문제가 있을 수 있습니다.
2. 증상을 먼저 관찰하고 원인을 추론합니다. 차이를 바로 확인하기보다 가설을 세우고 검증하는 과정이 문제 해결 능력을 키웁니다.
3. 이미지 내부 파일은 보안 문제와 연결됩니다. `.dockerignore` 설정을 확인하고 민감한 정보가 이미지에 들어가지 않도록 해야 합니다.
4. 복구와 재검증까지 해야 문제 해결이 끝납니다. 원인을 찾아 수정한 뒤 정상 동작을 확인하고, 실습 환경도 원래대로 되돌립니다.

한 문장 요약

Dockerfile의 작은 변경도 기능·보안·실행 환경·접속 가능성에 영향을 줄 수 있으므로, 증상으로 원인을 추론하고 검증한 뒤 복구하는 훈련이 중요합니다.

Chapter07

/강의정리 챕터07. 장애 대응 훈련 다시 하기

1. 학습 목적

이번 수업은 앞서 배운 Docker 이미지 관리, 배포 추적, 이미지 승격, 롤백, 정리 정책을 실제 장애 대응 훈련으로 다시 확인하는 과정입니다.

처음 훈련했을 때는 여러 화면을 찾아가며 답을 구해야 했다면, 이번에는 다음 세 가지 정보를 활용해 더 빠르게 판단하는 것이 목표입니다.

- production 서비스의 `/version` 응답
- Cloud Run 리비전 상세에 표시된 이미지 주소
- Artifact Registry `friend-agent` 저장소의 태그

핵심은 현재 서비스에서 실행 중인 코드가 무엇인지 확인하고, 문제가 생기면 어떤 이미지로 되돌아갈 수 있는지 판단하는 능력입니다.

2. 중학생도 이해할 수 있는 설명

프로그램 배포를 책을 출판하는 과정에 비유해 보겠습니다.

커밋 SHA — 원고의 고유 기록

어떤 소스 코드 버전으로 프로그램을 만들었는지 알려 줍니다.

이미지 태그 — 책에 붙인 이름표

커밋 SHA를 태그로 사용하면 특정 버전의 이미지를 찾기 쉽습니다. 다만 태그는 다른 이미지로 옮겨 붙을 수 있습니다.

다이제스트 — 책의 실제 지문

이미지 내용에 따라 정해지는 식별값입니다. 실제로 어떤 이미지가 배포되었는지 확인할 때 사용합니다.

Cloud Run 리비전 — 실제 서비스에 올라간 판본

해당 리비전이 어떤 이미지로 실행되는지 확인할 수 있습니다.

이 네 가지 정보가 연결되어 있으면 "지금 서비스에서 어떤 코드가 실행되고 있는가?"라는 질문에 빠르게 답할 수 있습니다.

3. 오늘 복습한 핵심 개념

개념	이번 수업에서 확인한 내용
이미지 이름과 태그	저장소 위치와 커밋 SHA를 이용해 이미지를 찾습니다.
다이제스트	실제 이미지의 내용을 식별합니다.
커밋에서 리비전까지	커밋, 이미지, 리비전, `/version` 정보를 연결합니다.
이미지 승격	staging에서 검증한 이미지를 다시 빌드하지 않고 production에 배포합니다.
롤백	이전 이미지가 저장소에 남아 있어야 이전 버전으로 돌아갈 수 있습니다.
정리 정책	저장 공간을 줄이되 롤백에 필요한 이미지는 보존합니다.
Dockerfile 진단	증상과 로그를 근거로 원인을 추론하고 정상 상태로 복구합니다.

4. 이미지 승격이 중요한 이유

처음에는 staging과 production이 각각 이미지를 만들었습니다. 같은 소스 코드에서 출발했더라도 빌드 환경이나 시점에 따라 결과 이미지의 다이제스트가 달라질 수 있습니다.

이를 해결하기 위해 staging에서 검증한 이미지 자체를 production으로 승격하도록 구조를 바꾸었습니다.

GitHub main에 머지

커밋 SHA로 버전 식별

GitHub Actions에서 이미지 빌드

배포용 이미지는 Actions에서 생성

staging에서 검증

기능과 버전 확인

동일한 다이제스트로 production 배포

다시 빌드하지 않고 검증된 이미지 사용

이 구조의 장점은 staging에서 확인한 이미지와 production에서 실행되는 이미지가 동일하다는 점입니다. 강의에서 다룬 승격 워크플로를 적용하면 두 환경의 다이제스트가 같게 유지됩니다.

다만 이미지가 같다는 것과 배포 과정이 안전하다는 것은 별개의 문제입니다. staging에서 검증하는 동안 다른 변경이 들어오는 상황을 방지하거나 확인하는 절차도 필요합니다.

5. 주관식 퀴즈 답변 예시

질문은 두 가지입니다.

1. 세 질문 가운데 걸린 시간이 가장 많이 줄어든 질문과 그 이유
2. 오늘 구조를 바꾼 뒤에도 사람이 직접 해야 하는 일 한 가지

실제 훈련 시간 기록은 제공되지 않았으므로, 어느 질문에서 시간이 가장 많이 줄었는지는 확정할 수 없습니다. 다음은 강의 내용을 바탕으로 작성한 답변 예시입니다.

시간이 가장 많이 줄어든 질문은 실제 훈련 시간 기록을 비교해 판단해야 하지만, 커밋과 이미지의 연결을 확인하는 질문은 시간이 줄어들기 쉬운 항목입니다. 이전에는 여러 화면과 연결표를 확인해야 했지만, 이제는 `/version`의 커밋 SHA와 이미지 태그를 비교해 현재 실행 버전을 빠르게 확인할 수 있기 때문입니다.

오늘 구조를 바꾼 뒤에도 사람이 직접 해야 하는 일은 staging에서 검증한 버전이 승격 대상과 일치하는지 확인하는 것입니다. staging에 새로운 변경이 들어오면 검증하지 않은 이미지가 production으로 승격될 수 있으므로, 승인 전에 버전을 다시 확인해야 합니다.

6. 아직 해결해야 할 과제

이번 수업의 마지막 부분에서는 시스템이 개선되었지만 아직 자동화되지 않은 작업도 확인했습니다.

- 버전 확인: staging에서 검증한 이미지가 승격 대상인지 승인 전에 확인해야 합니다.
- 롤백: 이전 리비전이나 이미지를 직접 찾아 되돌리는 절차가 남아 있습니다.
- 정리 정책: 드라이런 결과를 확인한 뒤 실제 삭제 정책으로 전환해야 합니다.
- 배포 기록: 커밋, 리비전, 다이제스트 연결표를 사람이 채우고 있습니다.
- 배포 중 장애 감지: 새 리비전으로 바뀌는 순간 사용자가 오류를 경험하는지 아직 측정하지 않았습니다.

이 목록은 앞으로 자동화할 작업을 정하는 출발점입니다.

7. 학습 및 1인 AI 사업과의 연결

AI 에이전트를 직접 만들거나 고객사에 AI 서비스를 구축해 드릴 때도 기능을 구현하는 것만으로는 충분하지 않습니다.

서비스를 운영하려면 다음 질문에 답할 수 있어야 합니다.

- 지금 실행 중인 버전은 무엇인가?
- 문제가 생기면 이전 버전으로 돌아갈 수 있는가?
- 배포된 이미지가 테스트한 이미지와 같은가?
- 불필요한 자원을 정리하면서 복구에 필요한 자원은 보존하고 있는가?

- 장애가 발생했을 때 누가 무엇을 확인해야 하는가?

이 질문에 체계적으로 답할 수 있다면, 단순히 AI 서비스를 만드는 것을 넘어 안정적으로 운영하고 관리하는 컨설팅 역량을 갖추는 데 도움이 됩니다.

8. 반드시 기억할 것

- 태그는 이미지를 찾기 위한 이름이며, 다이제스트는 이미지 내용을 식별합니다.
- 커밋 SHA, 이미지, 리비전, `/version` 정보를 연결하면 현재 실행 버전을 추적할 수 있습니다.
- staging에서 검증한 이미지를 동일한 다이제스트로 production에 승격하면 환경 간 이미지 차이를 줄일 수 있습니다.
- 롤백하려면 이전 리비전뿐 아니라 해당 이미지도 보존되어 있어야 합니다.
- 자동화 이후에도 검증된 버전인지 확인하고, 장애 복구와 배포 안전성을 점검하는 절차는 필요합니다.

한 문장 요약: 이번 수업은 이미지와 배포 이력을 연결해 장애 원인을 더 빠르게 찾고, 검증된 이미지를 안전하게 승격하며, 아직 남은 운영 작업을 구체적으로 파악하는 훈련입니다.